

Отлично, рассмотрю сущность План критически и детально.

Ran tool

Ran tool

Ran tool

Ran tool

Read file: src/economy_a5/models/core.py

Отлично, теперь критически проанализирую сущность План и её поведение.

Сущность Plan: Детальная архитектура

Базовая структура

```
@dataclass
class Plan:
    id: str
    name: str
    type: PlanType
    parent_plan_id: Optional[str]

    # Конфигурация поведения
    config: PlanConfig

    # Правила автоматики
    fill_rules: List[FillRule]
    spend_rules: List[SpendRule]

    # Приоритет для конкуренции за ресурсы
    priority: int

    # Временные рамки
    created_at: datetime
    archived_at: Optional[datetime]

    enum PlanType:
        FUND          # Накопительный фонд
        GOAL           # Цель с датой/суммой
        EXPENSE_BUDGET # Бюджет расходов
        INCOME_SOURCE  # Источник доходов
        STORAGE        # Временное хранилище
```

Критическая проблема №1: Балансы во времени

Проблема: План должен показывать баланс в любой момент времени (прошлый, текущий, будущий).

Решение: Событийная модель + кеширование

```

@dataclass
class PlanEvent:
    plan_id: str
    timestamp: datetime
    amount: Decimal
    event_type: EventType # INCOME, EXPENSE, TRANSFER_IN, TRANSFER_OUT
    source: str           # Откуда событие: "transaction:123", "rule:auto_fill",
    "manual"
    scenario: ScenarioType = REALISTIC

@dataclass
class PlanBalance:
    plan_id: str
    date: date
    balance: Decimal
    scenario: ScenarioType

    # Кеш для быстрого доступа, пересчитывается из событий

```

Поведение по типам:

1. **FUND:** Баланс накапливается, может быть отрицательным (долг фонду)
2. **GOAL:** Баланс растет до target_amount, после достижения остается
3. **EXPENSE_BUDGET:** Баланс уменьшается при тратах, пополняется по правилам
4. **INCOME_SOURCE:** Баланс обычно 0, генерирует события пополнения других планов
5. **STORAGE:** Простое хранилище, может опустошаться полностью

Критическая проблема №2: Правила миграции денег

FillRule - как деньги попадают в план:

```

@dataclass
class FillRule:
    source_type: SourceType # INCOME_CATEGORY, OTHER_PLAN, SURPLUS,
    TRANSACTION
    source_filter: str      # "category:salary" или "plan:main_fund"

    allocation_type: AllocationType # PERCENTAGE, FIXED_AMOUNT, REMAINDER
    allocation_value: Decimal       # 20.0 для 20% или 500.0 для 500$

    schedule: Optional[Schedule] # Когда применять: "monthly", "weekly", None
    для мгновенно
    priority: int                 # Порядок применения правил

    # Условия активации
    conditions: List[Condition]   # "min_balance > 1000", "date > 2024-01-01"

```

SpendRule - как деньги уходят из плана:

```
@dataclass
class SpendRule:
    target_type: TargetType          # CATEGORY, OTHER_PLAN, EXTERNAL
    target_filter: str                # "category:food" или "plan:emergency"

    spend_type: SpendType            # AUTO_COVER, BUDGET_LIMIT, GOAL_TARGET
    spend_value: Optional[Decimal]   # Лимит или null для полного покрытия

    schedule: Optional[Schedule]     # Периодичность трат
    priority: int
```

Критическая проблема №3: Иерархия планов

Решение: Дочерние планы как "подсчета" родительского

```
def get_effective_balance(plan: Plan, date: date, scenario: ScenarioType) ->
Decimal:
    """Баланс плана = собственный баланс + сумма балансов дочерних планов"""

    own_balance = get_direct_balance(plan.id, date, scenario)

    if plan.config.aggregate_children:
        children_balance = sum(
            get_effective_balance(child, date, scenario)
            for child in get_child_plans(plan.id)
        )
        return own_balance + children_balance

    return own_balance
```

Правило: Дочерний план может "занимать" у родительского, создавая отрицательный баланс.

Критическая проблема №4: Связь с транзакциями

PlanTransaction - мягкая связь:

```
@dataclass
class PlanTransaction:
    transaction_id: str
    plan_id: str
    amount: Decimal                # Может отличаться от суммы транзакции
    allocation_type: AllocationType # AUTO, MANUAL, SUGGESTED
    confidence: float              # 0.0-1.0, уверенность автоматического
    СВЯЗЫВАНИЯ
```

Автоматическое связывание:

```
class TransactionMatcher:
    def suggest_plan(self, transaction: Transaction) -> List[PlanSuggestion]:
        """Предлагает планы для транзакции на основе правил"""

        # 1. Точное совпадение по SpendRule
        # 2. Совпадение по категории
        # 3. Совпадение по сумме/паттерну
        # 4. Дефолтный план "Нераспределенное"
```

Критическая проблема №5: Проекция в будущее

TimeProjection - расчет будущих состояний:

```
@dataclass
class ProjectionEngine:
    def calculate_projections(
        self,
        plans: List[Plan],
        from_date: date,
        to_date: date,
        scenario: ScenarioType
    ) -> List[PlanBalance]:
        """
        Алгоритм:
        1. Берем текущие балансы планов
        2. Для каждого дня в диапазоне:
            - Применяем все FillRule (доходы, трансферы)
            - Применяем все SpendRule (расходы)
            - Применяем правила перераспределения излишков
            - Сохраняем снимок балансов
        3. Возвращаем временной ряд
        """
```

Критическая проблема №6: Сценарии планирования

```
@dataclass
class ScenarioModifier:
    target_type: str          # "income", "expense", "plan:emergency"
    modifier_type: ModifierType # MULTIPLY, ADD, REPLACE
    value: Decimal            # 0.8 для -20%, 100 для +100$

@dataclass
class Scenario:
    name: str
    base_scenario: ScenarioType = REALISTIC
    modifiers: List[ScenarioModifier]
```

Управление простотой

1. Умные дефолты:

- Новый план автоматически получает базовые правила
- Категории транзакций автоматически линкуются к планам
- Приоритеты планов устанавливаются автоматически

2. Автоматическое поведение:

```
# При создании плана "Продукты"
plan = Plan(name="Продукты", type=EXPENSE_BUDGET)

# Автоматически создаются правила:
# SpendRule: покрывать категорию "food"
# FillRule: 15% от зарплаты (базовая эвристика)
```

3. Предотвращение отрицательных балансов:

```
def validate_transaction(transaction: Transaction, target_plan: Plan) ->
ValidationResult:
    """Проверяет, не уйдет ли план в критический минус"""

    projected_balance = calculate_balance_after(target_plan, transaction)

    if projected_balance < target_plan.config.critical_minimum:
        return ValidationResult(
            valid=False,
            suggestion="Перевести средства из плана '{suggested_source}'"
        )
```

Нерешенные проблемы и варианты

Проблема 1: Как обрабатывать изменения правил во времени?

Варианты:

- Правила действуют с момента создания
- Версионирование правил с датами начала действия
- Пересчет всех проекций при изменении правил

Выбор: Правила действуют с момента создания, пересчет проекций при изменениях.

Проблема 2: Как обрабатывать конкуренцию правил за одни деньги?

Решение: Система приоритетов + валидация при создании правил.

Проблема 3: Производительность расчета проекций для больших временных диапазонов.

Решение: Кеширование ежемесячных агрегатов + инкрементальный пересчет.

Эта архитектура покрывает все описанные сценарии, сохраняя простоту использования через автоматизацию и умные дефолты.